

RISC-V Simulator Implementation

과목명: 컴퓨터구조

개발 환경: Windows 11 / CLion / GCC (MinGW)

1. 개요

본 과제는 32비트 기계어(Machine Code)로 작성된 입력 파일을 읽어들이는 RISC-V 시뮬레이터를 구현하는 것을 목표로 한다. 시뮬레이터는 크게 두 가지 기능을 수행한다. 첫째, 기계어를 분석하여 대응하는 RISC-V 어셈블리 코드(.s)로 변환(Translation)한다. 둘째, 해당 명령어들을 실제로 실행(Simulation)하여 프로그램 종료 시점의 레지스터 상태와 PC(Program Counter) 값을 출력한다.

2. 구현 환경 및 실행 방법

본 프로젝트는 C언어(C99 표준)를 사용하여 구현되었으며, Windows 환경의 CLion IDE에서 개발 및 테스트를 진행하였다.

컴파일: 별도의 옵션 없이 기본 GCC 명령어로 컴파일 가능하다.

```
gcc -o main main.c
```

실행: 실행 후 입력 파일명을 요구하는 프롬프트에 파일명(예: test1.in)을 입력한다.

종료: terminate를 입력하면 프로그램이 종료된다.

3. 프로그램 구조 및 알고리즘

프로그램은 입력된 기계어 파일 전체를 메모리(instructionBuffer)에 로드한 뒤, 3단계(Pass)에 걸쳐 처리하는 구조로 설계되었다. 이는 분기(Branch) 명령어의 레이블(Label) 처리를 명확히 하고, 변환과 실행의 로직을 분리하여 안정성을 높이기 위함이다.

[Step 1] 레이블 위치 계산

Branch 명령어(SB-Type)는 오프셋(Offset)을 사용하여 점프할 위치를 지정한다. 어셈블리 코드에서는 이를 label1, label2와 같은 레이블로 표현해야 한다.

따라서, 실제 변환을 수행하기 전에 전체 코드를 스캔하여 Branch 명령어가 가리키는 목

적지 인덱스를 계산하고, 해당 위치에 레이블 번호를 부여하여 lineLabels 배열에 저장한다.

중복 처리: 서로 다른 분기문이 같은 위치를 가리킬 경우, 새로운 레이블을 생성하지 않고 기존 레이블 번호를 공유하도록 구현하였다.

[Step 2] 어셈블리 변환 및 파일 생성

입력 파일명에서 확장자를 제거하고 .s를 붙여 출력 파일을 생성한다. 각 명령어를 순회하며 다음 과정을 수행한다.

레이블 출력: 현재 줄에 Step 1에서 할당된 레이블이 있다면 labelN: 형식으로 먼저 출력한다.

명령어 파싱: 32비트 문자열을 파싱하여 Opcode, rd, rs1, rs2, funct3, funct7, imm 등의 필드로 분리한다.

Type별 처리: switch-case 문을 사용하여 R, I, S, SB 타입을 구분하고, 각 명령어 이름(Mnemonic)을 매핑한다.

에러 처리: 정의되지 않은 Opcode나 잘못된 포맷(범위 밖 점프, 정렬 오류 등)이 발견되면 즉시 파일을 삭제하고 에러 메시지를 출력한다.

[Step 3] 명령어 실행

가상의 CPU 구조체(regs[32], pc)를 선언하고 초기화한다. PC 값을 기준으로 명령어를 인출(Fetch)하고 실행(Execute)하는 루프를 수행한다.

PC 업데이트: 기본적으로 $PC += 4$ 를 수행하되, Branch 조건이 참인 경우 $PC += \text{imm}$ 으로 점프한다.

종료 조건: PC가 명령어 영역을 벗어나면(instCount 초과) 프로그램을 종료하고 최종 레지스터 값을 출력한다.

4. 주요 구현 상세

4.1. 비트 파싱 및 데이터 변환 (Bit Parsing)

입력 파일로부터 읽어들이는 32자리의 문자열('0'과 '1'의 조합)을 실제 정수 데이터로 변환

하기 위해 별도의 파싱 함수(parseBinary)를 구현하여 사용하였다. 이 함수는 전체 명령어 문자열에서 필요한 구간의 시작 인덱스와 길이를 인자로 받아, 해당 부분 문자열(Substring)을 추출한 뒤 strtol 함수를 이용해 10진수 정수로 변환하여 반환한다.

특히 RISC-V 명령어 포맷에서 각 필드(Opcode, rd, rs1 등)의 비트 위치는 우측 끝이 0번 비트인 반면, C언어의 문자열 배열 인덱스는 좌측 끝이 0번부터 시작한다는 점을 고려하여 인덱싱을 수행하였다. 예를 들어, 최상위 비트(MSB, 31번 비트)는 문자열의 0번 인덱스에 해당하며, Opcode(0~6번 비트)는 문자열의 25~31번 인덱스에 위치하도록 매핑하여 정확한 값을 추출하였다.

4.2. 부호 확장

I-Type 및 SB-Type 명령어의 Immediate 값은 음수일 수 있다. 따라서 12비트(또는 13비트) 값을 32비트 정수로 변환할 때, 최상위 비트(MSB)가 1인 경우 나머지 상위 비트를 모두 1로 채우는 부호 확장을 수행하였다.

I-Type의 경우 (12비트)

```
if ((imm >> 11) & 1) imm |= 0xFFFFF000;
```

SB-Type의 경우 (13비트)

```
if ((imm >> 12) & 1) imm |= 0xFFFFE000;
```

4.3. 에러 처리 및 예외 상황

입력 검증: 파일 내에 0과 1 이외의 문자가 포함되어 있거나, 32글자가 아닌 경우 에러 처리하였다.

파일 정리: 포맷 에러가 발생한 경우, 생성 중이던 .s 파일을 remove() 함수로 삭제하여 잘못된 파일이 남지 않도록 하였다.

오버플로우 테스트: test2.in과 같이 수십억 번의 반복을 수행하는 경우에도 시뮬레이터가 중단되지 않고 정상적으로 연산을 마칠 때까지 수행하도록 구현하였다.

5. 실행 결과

test1.in, test2.in, test3.in 각각 실행 결과 출력 화면 과 그에 따라 생성된 파일

```
Run main.c x
C:\Users\danawa\Desktop\DEV\CP\RISCV_Simulator\main.exe
>> Enter Input File Name:test1.in
>> Final values in PC, x1, x2, x3, x4, x5 are 1076, 1, 2, 3, 4, 5
>> Enter Input File Name:test3.in
>> Instruction Format Error!
>> Enter Input File Name:test2.in
>> Final values in PC, x1, x2, x3, x4, x5 are 1044, 1, 1, 3, 9, 5
>> Enter Input File Name:|
```

```
test1.s x
1 ADD x7, x1, x2
2 SUB x8, x3, x4
3 AND x9, x5, x6
4 OR x10, x7, x8
5 XOR x11, x9, x10
6 ANDI x12, x11, 11
7 ORI x13, x12, 15
8 XORI x14, x13, 12
9 SLL x15, x14, x2
10 SRL x16, x15, x3
11 SRA x17, x16, x4
12 SLLI x18, x17, 2
13 SRLI x19, x18, 1
14 SRAI x20, x19, 1
15 LW x21, 0(x1)
16 ADD x22, x1, x2
17 SUB x23, x2, x1
18 AND x24, x22, x23
19 OR x25, x22, x23
20 SW x21, 8(x2)
```

```
test2.s x
1 ADD x2, x1, x0
2 SUB x8, x7, x1
3 ADDI x4, x0, 9
4 ORI x9, x8, 45
5 label1:
6 ADD x7, x7, x8
7 SUB x9, x9, x8
8 BGE x9, x8, label1
9 ADDI x9, x0, 5
10 ADDI x7, x0, 7
11 BGE x7, x9, label2
12 ADD x10, x1, x2
13 label2:
14 SUB x11, x3, x4
```

6. 결론 및 고찰

본 과제를 수행하며 기계어와 어셈블리어 간의 1:1 매핑 구조와 분기 명령어의 주소 계산 방식을 명확히 이해할 수 있었다. 특히 test2.in 테스트 케이스는 정수 오버플로우가 발생할 때까지 약 21억 번 이상의 명령어를 수행하는 고부하 작업이었는데, 노트북 환경에서는 응답이 지연되었으나 데스크탑 환경에서는 정상적으로 결과가 출력되는 현상을 관찰하였다.

이는 소프트웨어 시뮬레이터가 1개의 RISC-V 명령어를 처리하기 위해 수십 개의 호스트 명령어를 실행해야 하는 시뮬레이션 오버헤드(Overhead)와, 호스트 프로세서의 클럭 속도(Clock Rate) 차이에 기인한 결과로 분석된다. 이를 통해 단순히 알고리즘의 논리적 정확성뿐만 아니라, 실행 환경의 컴퓨팅 파워가 대규모 연산 시뮬레이션의 실행 시간 (Execution Time)에 결정적인 영향을 미친다는 컴퓨터 구조의 성능 원리를 몸소 체감할 수 있었다.